

# cron4j 2.2 官方手册（中文完整版）

本文档已完整翻译 cron4j 2.2 官方手册原文，保留代码示例、语法规则与结构，可直接导出为 PDF 离线查阅。

---

## 目录

1. 快速入门
  2. 调度表达式
  3. 任务的调度、重调度与取消调度
  4. 调度系统进程
  5. 从文件调度进程
  6. 自定义任务
  7. 自定义任务收集器
  8. 自定义调度器监听器
  9. 执行器
  10. 手动启动任务
  11. 时区设置
  12. 守护线程
  13. 执行时间预测器
  14. Cron 解析器
- 

## 1. 快速入门

cron4j 的核心组件是**调度器（Scheduler）**。通过 `it.sauronsoftware.cron4j.Scheduler` 实例，可全年在指定时刻执行任务。调度器支持每分钟、每 5 分钟、周五 10:00、2 月 16 日 12:30（且仅当当天是周六）等多种触发规则。

使用 cron4j 调度器分四步：

1. 创建 Scheduler 实例
2. 调度任务：指定**执行内容**（Runnable/Task）与**执行时间**（调度表达式字符串 / SchedulingPattern 对象）

### 3. 启动调度器

### 4. 不再使用时停止调度器

## 示例代码

代码块

```
1  import it.sauronsoftware.cron4j.Scheduler;
2
3  public class Quickstart {
4      public static void main(String[] args) {
5          // 创建调度器实例
6          Scheduler s = new Scheduler();
7          // 调度每分钟执行一次的任务
8          s.schedule("* * * * *", new Runnable() {
9              public void run() {
10                 System.out.println("又一分钟过去了.....");
11             }
12         });
13         // 启动调度器
14         s.start();
15         // 运行10分钟
16         try {
17             Thread.sleep(1000L * 60L * 10L);
18         } catch (InterruptedException e) {
19             // 忽略中断
20         }
21         // 停止调度器
22         s.stop();
23     }
24 }
```

## 核心特性

- 可调度任意数量任务
- 调度器运行中仍可新增 / 修改 / 删除任务
- 支持重调度（修改已有任务的执行时间）
- 支持取消调度（移除任务）
- 调度器可多次启停
- 支持从文件 / 自定义源加载任务
- 支持任务执行事件监听
- 支持手动触发任务

- 支持时区切换
- 支持表达式预校验
- 支持预测任务执行时间

## 2. 调度表达式

类 UNIX crontab 格式，由**5 个空格分隔的字段**组成，依次为：

| 字段    | 含义 | 取值范围          | 特殊值           |
|-------|----|---------------|---------------|
| 第 1 位 | 分钟 | 0-59          | —             |
| 第 2 位 | 小时 | 0-23          | —             |
| 第 3 位 | 日期 | 1-31          | L（当月最后一天）     |
| 第 4 位 | 月份 | 1-12          | jan/feb/…/dec |
| 第 5 位 | 星期 | 0 (周日)-6 (周六) | sun/mon/…/sat |

`*` 为通配符，表示**每一个**匹配单位。  
当 5 个字段**同时满足**当前时间时，任务触发执行。

非法表达式会抛出 `it.sauronsoftware.cron4j.InvalidPatternException`，可通过 `SchedulingPattern.validate(String)` 预校验。

### 常用示例

- `5 * * * *`：每小时第 5 分钟执行一次（00:05、01:05…）
- `* * * * *`：每分钟执行
- `* 12 * * Mon`：周一 12 点整的每一分钟
- `* 12 16 * Mon`：每月 16 日且为周一的 12 点整每一分钟
- `59 11 * * 1-5`：周一至周五 11:59 执行
- `*/5 * * * *`：每 5 分钟执行
- `3-18/5 * * * *`：每小时第 3、8、13、18 分钟执行
- `*/15 9-17 * * *`：9 点-17 点每 15 分钟执行（最后一次 17:45）
- `0 5 * * *|8 10 * * *|22 17 * * *`：每天 05:00、10:08、17:22 各执行一次

## 3. 任务的调度、重调度与取消调度

### 任务创建方式

1. 实现 `java.lang.Runnable` 接口（最简单）
2. 继承 `it.sauronsoftware.cron4j.Task` 抽象类（功能更强）

### 核心方法

- `schedule(表达式, Runnable/Task)`：调度任务，返回**任务 ID**
  - `reschedule(任务ID, 新表达式)`：重调度（修改执行时间）
  - `deschedule(任务ID)`：取消调度（移除任务）
  - `getSchedulingPattern(任务ID)`：获取任务当前表达式
  - `getTask(任务ID)`：获取任务对象
- 

## 4. 调度系统进程

使用 `ProcessTask` 可直接调度外部系统进程（如 exe、bat、shell 等）。

### 基础用法

代码块

```
1 ProcessTask task = new ProcessTask("C:\\Windows\\System32\\notepad.exe");
2 Scheduler scheduler = new Scheduler();
3 scheduler.schedule("* * * * *", task);
4 scheduler.start();
```

### 带参数 / 环境变量 / 工作目录

代码块

```
1 String[] command = { "C:\\tomcat\\bin\\catalina.bat", "start" };
2 String[] envs = { "CATALINA_HOME=C:\\tomcat", "JAVA_HOME=C:\\jdfs\\jdk5" };
3 File directory = new File("C:\\MyDirectory");
4 ProcessTask task = new ProcessTask(command, envs, directory);
```

### 重定向输入输出

代码块

```
1 task.setStdinFile(new File("in.txt")); // 标准输入
2 task.setStdoutFile(new File("out.txt")); // 标准输出
3 task.setStderrFile(new File("err.txt")); // 错误输出
```

---

## 5. 从文件调度进程

可加载类 crontab 格式文件批量调度进程：

- `scheduleFile(File)`：加载并调度文件中的任务
- `descheduleFile(File)`：取消该文件的调度
- `getScheduledFiles()`：获取已加载的调度文件

调度器**每分钟解析一次文件**，匹配时间则执行对应任务。

---

## 6. 自定义任务

继承 `Task` 并实现 `execute(TaskExecutionContext)`，可获得更强控制能力：

### 能力清单

1. **状态跟踪**：重写 `supportsStatusTracking()` 返回 true，用 `setStatusMessage()` 同步状态
  2. **进度跟踪**：重写 `supportsCompletenessTracking()` 返回 true，用 `setCompleteness(0-1)` 同步进度
  3. **暂停 / 恢复**：重写 `canBePaused()` 返回 true，循环调用 `pauseIfRequested()`
  4. **停止**：重写 `canBeStopped()` 返回 true，循环判断 `isStopped()` 并安全退出
  5. 获取调度器 / 执行器：`context.getScheduler()` / `context.getTaskExecutor()`
- 

## 7. 自定义任务收集器

通过 `TaskCollector` 可接入**外部任务源**（数据库、远程配置、XML 等）：

- `addTaskCollector(TaskCollector)`：注册收集器
- `removeTaskCollector(TaskCollector)`：移除收集器
- `getTaskCollectors()`：获取已注册收集器

调度器**每分钟调用一次** `getTasks()`，返回 `TaskTable`（任务 + 表达式映射），自动执行匹配任务。

---

## 8. 自定义调度器监听器

实现 `SchedulerListener` 监听调度器生命周期：

- `taskLaunching(TaskExecutor)`：任务即将启动
- `taskSucceeded(TaskExecutor)`：任务执行成功
- `taskFailed(TaskExecutor, Throwable)`：任务执行失败

### 使用方式

代码块

```
1 scheduler.addSchedulerListener(你的监听器);  
2 scheduler.removeSchedulerListener(你的监听器);
```

---

## 9. 执行器 (TaskExecutor)

执行器类似线程，负责运行任务：

- `getExecutingTasks()`：获取当前正在执行的任务列表
- `isAlive()`：是否正在运行
- `join()`：等待任务执行完成
- `pause()` / `resume()`：暂停 / 恢复（任务需支持）
- `stop()`：停止任务（任务需支持）
- `getStatusMessage()`：获取任务状态文本
- `getCompleteness()`：获取进度（0-1）

同时支持 `TaskExecutorListener` 监听执行器事件（暂停、恢复、停止、结束、状态 / 进度变更）。

---

## 10. 手动启动任务

调度器运行中，可不通过表达式直接触发任务：

代码块

```
1 TaskExecutor executor = scheduler.launch(task);
```

---

返回执行器，可实时控制与监控任务。

---

## 11. 时区设置

调度器默认使用**系统时区**，可指定自定义时区：

- `setTimeZone(TimeZone)`：设置时区
- `getTimeZone()`：获取当前时区

设置后，系统时间会先转为该时区再匹配表达式。

---

## 12. 守护线程

调度器可配置为**仅生成守护线程**：

- `setDaemon(boolean)`：必须在启动前调用
- `isDaemon()`：查看是否为守护模式

JVM 在只剩守护线程时会自动退出。

---

## 13. 执行时间预测器

`Predictor` 可预测表达式未来触发时间：

代码块

```
1 String pattern = "0 3 * jan-jun,sep-dec mon-fri";
2 Predictor p = new Predictor(pattern);
3 // 打印未来n次执行时间
4 for (int i = 0; i < n; i++) {
5     System.out.println(p.nextMatchingDate());
6 }
```

---

## 14. Cron 解析器

`CronParser` 用于解析类 crontab 格式内容（流 / 字符串），适合非文件来源（DB、远程接口）。

### 有效行格式

调度表达式 [选项] 命令 [参数]

## 支持选项

- `IN:文件路径`：重定向标准输入
- `OUT:文件路径`：重定向标准输出
- `ERR:文件路径`：重定向错误输出
- `ENV:NAME=VALUE`：设置环境变量
- `DIR:路径`：设置工作目录

## Java 方法调用语法

调度表达式 `java:全类名#方法名 参数`

省略方法名默认调用 `main(String[])`，在当前 JVM 内直接执行，不支持进程相关选项。

---

## 导出为 PDF 指南（Windows/macOS 通用）

1. 复制本文档全部内容到**Word/WPS/ 记事本**
  2. 排版：标题分级、代码块等宽字体、保留目录
  3. 执行导出：
    - Windows：文件 → 另存为 → 选择 PDF
    - macOS：文件 → 打印 → 存储为 PDF
  4. 保存到本地即可离线使用
- 

需要我帮你把这份中文手册**整理成标准 A4 排版、带书签目录**的可直接导出 PDF 的纯文本吗？

（注：文档部分内容可能由 AI 生成）